

MYWEDDINGDAY · ENGINEERING NOTES

System Design Concepts Behind This Project

A wedding/pre-wedding/band-booking marketplace (Angular + Express + PostgreSQL) that started as a straightforward CRUD app and had a deliberate second pass added: caching, reliability patterns, observability, and a handful of architecture decisions worth being able to explain out loud. This document is the "why," with exact file references for the "where."

STACK Angular 19 · Express · PostgreSQL 16 ·
Redis 7

SCOPE Caching · Reliability · Observability ·
Architecture

PURPOSE Interview-ready talking points, grounded
in real running code

CONTENTS

1. Caching & Redis
2. Reliability
3. Observability
4. Architecture talking points
5. How to talk about this in an interview

1 · Caching & Redis

Redis is optional infrastructure everywhere it's used here — every helper degrades to "just hit Postgres" if Redis is unset or unreachable. That degradation is itself the first talking point: a cache should never be a single point of failure for correctness, only for speed.

1.1 Cache-aside pattern

On a read, check Redis first. Miss → query Postgres, write the result into Redis with a TTL, then return it. On a write, invalidate the affected keys directly instead of waiting for the TTL. This is the most common caching pattern in real systems (as opposed to write-through or write-behind) because the application stays in full control of what gets cached and for how long, and a cache outage just means every request falls through to the database — slower, not broken.

IMPLEMENTATION

Applied to the four public read endpoints most likely to be hit repeatedly: `GET /api/venues`, `/api/venues/:id`, `/api/bands`, `/api/bands/:id`, and `GET /api/reviews`. 30-second TTL. Every write endpoint (create/update/delete venue or band, post/edit/delete a review, the Stripe webhook completing a payment) explicitly deletes the relevant cache keys rather than relying on the TTL alone — the TTL exists as a safety net for any invalidation path that gets missed, not as the primary consistency mechanism.

```
backend/src/config/redis.js - cacheGet / cacheSet / cacheInvalidate
backend/src/routes/venues.js, bands.js, reviews.js - wired into each GET/write handler
Responses carry an X-Cache: HIT / MISS header - directly observable, not just claimed
```

SAY THIS

"I used cache-aside instead of write-through because the read/write ratio here is read-heavy and skewed toward a handful of popular listings — cache-aside means only data that's actually being read gets cached, rather than paying the cache-write cost on every write regardless of whether it's ever read again."

1.2 Rate limiting

A fixed-window counter per client (here: per IP) stored in Redis, shared across however many backend instances are running — the entire reason to centralize this in Redis instead of an in-memory `Map` is that in-memory counting is per-instance, so a client could bypass a single instance's limit just by hitting a different one behind the load balancer.

IMPLEMENTATION

`express-rate-limit` with a `rate-limit-redis` store. 20 requests / 15 minutes on `/api/auth/register`, `/login`, `/google` (the classic brute-force targets); 20 requests / minute on `POST /api/reviews` and `POST /api/payments/checkout` (cheap-to-spam write endpoints). Falls back to the library's built-in in-memory store if `REDIS_URL` isn't set.

```
backend/src/middleware/rateLimit.js
```

TRADEOFF DISCUSSED

Fixed-window counters are simple but allow a burst of up to 2× the limit right at a window boundary (e.g. 20 requests in the last second of one window, 20 more in the first second of the next). A sliding-window or token-bucket algorithm avoids that at the cost of a slightly more complex Redis data structure (a sorted set instead of a single counter key). Named here as a known, deliberate simplification — not something I didn't think about.

1.3 JWT revocation (session management)

JWTs are stateless by design — the whole appeal is that any server can verify one without a database lookup. The cost is that they're also *unrevokable* by design: a stolen or logged-out token stays valid until it naturally expires, because there's nothing to "delete." This app's JWTs last 7 days, so that's a real gap.

IMPLEMENTATION

Every signed token gets a random `jti` (JWT ID) claim. `POST /api/auth/logout` writes `revoked:jwt:<jti>` into Redis with a TTL equal to the token's remaining lifetime (no point keeping it around after the token would've expired anyway). The `authenticate` middleware checks that key on every request before trusting the token.

```
backend/src/routes/auth.js - signToken() adds jti, POST /logout blacklists it
backend/src/middleware/auth.js - checks the blacklist
web/src/app/core/services/auth.service.ts - logout() calls it (best-effort)
```

SAY THIS

"This is a hybrid approach — I kept JWTs stateless for the common case (no DB/Redis hit on every authenticated request), but added a small piece of shared state specifically for the one thing pure JWTs can't do: revocation. It's the same idea as a refresh-token denylist, just scoped to the access token itself since this app doesn't have separate refresh tokens."

1.4 Fail open vs. fail closed

Two Redis-backed features in this app, two different failure postures, on purpose:

FEATURE	IF REDIS IS DOWN	WHY
Cache-aside	Fails open — falls through to Postgres	A cache miss is just slower, never wrong
Rate limiting	Fails closed -ish — request errors rather than silently un-limiting	Silently disabling abuse protection when its store disappears defeats the purpose of having it

SAY THIS

"Not every dependency failure should be handled the same way — it depends what the dependency is protecting. I made that decision explicit and different in two places in the same codebase, rather than defaulting to one policy everywhere."

2 · Reliability

This app talks to two real third-party dependencies (Stripe, Google) and has one real money-moving endpoint (checkout). Reliability here means: don't double-charge, don't cascade a dependency's outage into this app's own outage, and don't lose the ability to change the schema safely once there's real data in it.

2.1 Idempotency keys

A double-click, a slow network causing a client retry, or a proxy replaying a request should never create two Stripe Checkout Sessions (and, downstream, two payment attempts) for the same intent. An idempotency key lets the payment provider recognize "I've seen this exact request before" and return the original result instead of doing the action again.

IMPLEMENTATION

Passed straight through to Stripe's own `idempotencyKey` option, built from the specific pending `Payments` row's id — so it's unique per checkout attempt and can never collide across users or listings, and a client-supplied `Idempotency-Key` header is honored if present.

```
backend/src/routes/payments.js
```

SAY THIS

"Idempotency matters most exactly where money moves. I didn't build a generic idempotency framework for every endpoint — I used the payment provider's own native support where it exists, which is simpler and more correct than reinventing it."

2.2 Circuit breaker

When a downstream dependency starts failing, the naive behavior — keep calling it, wait for each timeout — makes things worse: requests pile up holding connections/threads open, latency climbs everywhere, and the caller can tip over even though its own code is fine. A circuit breaker tracks recent failures and, once a threshold is crossed, "opens": further calls fail immediately (no network call at all) for a cooldown period, then allows one trial call through to check for recovery (half-open) before fully closing again.

IMPLEMENTATION

`opossum`, wrapping two calls: Stripe's `checkout.sessions.create` and Google's `verifyIdToken`. The Google one needed an extra refinement — most calls to `verifyIdToken` that fail do so because the token itself is invalid or expired, which is normal, expected, high-frequency traffic, *not* a sign Google's servers are unhealthy. Counting those toward the breaker would trip it from ordinary user error and lock out Google Sign-In for everyone. Opossum's `errorFilter` option solves this: only errors that look like genuine network failures (`ECONNREFUSED`, timeouts, etc.) count toward opening the breaker.

```
backend/src/utils/circuitBreaker.js - shared factory, wires up open/half-open/close  
logging  
backend/src/config/google.js - errorFilter distinguishing network faults from bad  
tokens  
backend/src/routes/payments.js - wraps the Stripe call, returns 503 while open
```

SAY THIS

"The interesting part wasn't wrapping the call — that's one line. It was realizing that a naive circuit breaker on the Google verification call would be actively harmful, because the overwhelming majority of its failures are expected user-side errors, not Google being down. Filtering which errors count is what makes it correct instead of just present."

2.3 Schema migrations

Ad-hoc `CREATE TABLE IF NOT EXISTS` on every boot is fine for a solo hobby project with no production data yet, but it has no concept of "change an existing column," no history of what changed when, and no rollback. A migrations tool tracks a linear, versioned, reversible history of schema changes as code.

IMPLEMENTATION

`node-pg-migrate`, with four migration files mirroring this project's actual schema history: initial tables → add Google auth columns → add Reviews → add Payments. Each has a working `down` (verified by actually running it against a throwaway database, not just written and assumed correct). Local dev is intentionally untouched — `database.js`'s auto-create-on-boot still exists, so `npm run dev` needs no extra step. The migrations are the production-grade path: point a fresh database (e.g. a new Railway Postgres instance) at `npm run migrate:up` instead of ever booting the app against it directly.

```
backend/migrations/*.js
backend/scripts/migrate.js - builds a connection string from the existing DB_* env
vars
npm run migrate:up / migrate:down / migrate:create
```

SAY THIS

"I kept both approaches deliberately, for different audiences — auto-create-on-boot for zero-friction local dev, and real versioned migrations for anything that looks like production. That's a conscious call about who each mechanism is for, not indecision."

3 · Observability

"It works on my machine" doesn't answer "why did request X fail at 2am for user Y" — that needs logs you can search, errors you get paged for, and metrics you can graph. Three different tools because they answer three different questions.

3.1 Structured logging

`console.log("Get venues error:", err)` produces a line of text a human can read in a terminal and a log aggregator basically cannot query. Structured logging emits JSON — each field (error message, request id, route, user id) is independently searchable and alertable once shipped to something like Datadog, CloudWatch, or Loki.

IMPLEMENTATION

`pino` — chosen over `winston` for being one of the fastest structured loggers for Node (matters on a hot path like a request handler), JSON in production, pretty-printed and colorized in dev via `pino-pretty` so it's still readable locally. `pino-http` auto-logs every request/response with duration and status. Every `console.error` / `console.log` in the backend replaced.

```
backend/src/config/logger.js - used throughout backend/src/routes/*.js
```

3.2 Error tracking

Logs answer "what happened" if you already know where to look. Error tracking answers "something just broke in production" before a user has to report it — it groups repeated occurrences of the same error, captures the stack trace and request context automatically, and can page someone.

IMPLEMENTATION

`@sentry/node`, initialized before Express even exists so it can auto-instrument from the start, with `Sentry.setupExpressErrorHandler(app)` registered after all routes. Entirely inert — zero calls to Sentry's servers — unless `SENTRY_DSN` is set, matching the same "safe until configured" pattern already used for Stripe and Google in this codebase.

```
backend/src/config/sentry.js · server.js
```

3.3 Metrics

Logs and errors are about individual events. Metrics are about trends and thresholds — "p99 latency has crept up over the last hour," "error rate just doubled," "cache hit rate dropped to 40%" — the kind of thing you'd put on a dashboard and set an alert on, not read line by line.

IMPLEMENTATION

`prom-client` : default Node process metrics (CPU, memory, event-loop lag) plus custom ones — `http_request_duration_seconds` (histogram), `http_requests_total`, and `cache_hits_total` / `cache_misses_total` derived straight from each response's own `X-Cache` header. Every metric is labeled by route *pattern* (`/api/venues/:id`) rather than the literal URL, which matters — labeling by the literal URL would create one time series per distinct venue ID ever requested, an unbounded-cardinality metric that would eventually overwhelm whatever's storing it. Exposed at `GET /metrics` in Prometheus's text exposition format.

```
backend/src/config/metrics.js · GET /metrics
```

SCOPE, STATED HONESTLY

This produces real, correctly-formatted metrics, but there's no actual Prometheus server scraping them or Grafana dashboard rendering them in this repo — that's always-on infrastructure a solo project doesn't need running 24/7 just to exist. The next step is pointing a Prometheus scrape config at `/metrics` and building a dashboard on top; worth saying exactly that in an interview rather than implying more is running than actually is.

4 · Architecture talking points

These are less "code I wrote" and more "decisions I can explain" — the kind of thing an interviewer asks about specifically to see if you understand the implications of what you built, not just that it runs.

4.1 API versioning

Every route is reachable at both its original path and under `/api/v1/ ...` — the same router instance mounted at two paths, so this added zero behavioral risk to what already existed. `/api/health` and `/metrics` are deliberately left unversioned, because they're operational endpoints, not product API surface that consumers version against.

SAY THIS

"A future breaking change gets its own v2 router mounted alongside v1, rather than changing v1's behavior underneath whatever's still calling it. The versioning exists so that decision is possible later, even though nothing has needed a v2 yet."

4.2 Horizontal scaling

The backend holds no server-side session state — every request carries its own JWT, and the only shared state introduced anywhere (cache, rate limit counters, revoked-token list) lives in Redis, not in the Node process's memory. That's precisely what makes horizontal scaling free: start a second instance behind a load balancer and it needs no coordination with the first, because neither one is holding state the other doesn't have access to.

SAY THIS

"If I'd put rate-limit counters in a plain in-memory Map instead of Redis, the app would still work fine with one instance — but the moment you add a second instance behind a load balancer, each one enforces its own separate limit, and the actual effective limit silently doubles. Centralizing that state in Redis is what makes scaling out a config change instead of a bug."

4.3 CAP theorem / consistency tradeoffs

Introducing a cache is a small, concrete CAP-theorem decision, not just an abstract one: it's an explicit choice of availability/latency over strict consistency, because a cached response can be up to the TTL's length stale. Postgres itself remains the single source of truth and is strongly consistent — the cache is a deliberately-lossy accelerator in front of it, not a second source of truth.

SAY THIS

"The 30-second TTL is the size of the consistency window I'm accepting in exchange for not hitting Postgres on every request. If this were, say, an account balance instead of a venue listing, I'd make a completely different call — probably no cache at all, or a much shorter TTL plus synchronous invalidation on every possible write path."

4.4 Fail open vs. fail closed, revisited

Worth restating at the architecture level, not just the Redis level: every dependency a system relies on eventually fails, and "what happens when it does" is a decision, not an accident. This project makes that decision differently for different dependencies — cache-aside fails open (Postgres directly), rate limiting fails closed-ish (protects against the abuse it exists to prevent), and Stripe/Google calls fail via a circuit breaker that fails fast rather than either silently succeeding or hanging.

5 · How to talk about this in an interview

The honest framing: this started as a plain CRUD marketplace app (Angular + Express + Postgres, JWT auth, Stripe payments) and had a deliberate second pass added specifically to practice and demonstrate the concepts that come up in system design interviews — on real, running code, not a diagram. That's a perfectly good thing to say out loud; it's more credible than pretending every decision here was forced by real production traffic.

What tends to land well in an interview, roughly in this order:

"I added Redis for three different things — caching, rate limiting, and JWT revocation — and made each one degrade differently when Redis is unavailable, on purpose, because they're protecting different things."

"The circuit breaker on Google's token verification needed an error filter, because most of its failures are just normal invalid tokens, not Google being down — naively wrapping it would've made the breaker trip from ordinary traffic."

"I kept the simple auto-create-on-boot schema setup for local dev, and added real versioned migrations as the production-grade path, rather than ripping one out for the other."

If asked "what would you do differently at real scale" — the honest answers are already written into this document as tradeoffs: a sliding-window rate limiter instead of fixed-window, versioned cache-key prefixes instead of pattern-scanning invalidation, an actual running Prometheus + Grafana instance instead of just a correctly-formatted `/metrics` endpoint. Naming the next step you'd take is usually more convincing than pretending the current version is the final one.